Raj: Yes, that's right. Passwords need to be at least 8 characters to make it harder for hacking algorithms to guess them.

You: So "secret" would be rejected because it has only 6 characters?

Raj: Correct.

You: So what about "password". Would that be acceptable?

Raj: No, we also said that we need at least 1 digit.

You: So we did. So "password1" would be OK?

Raj: No, actually that's still really easy to hack. It's a word from a dictionary: the digit at the end wouldn't slow down a hacking algorithm for very long. Random letters and punctuation marks make it a bit harder.

You: OK, so would "password1!" be OK? It has a number and an exclamation mark, and it has more than 8 characters.

Raj: No, like I said, using words from a dictionary like "password" is really bad. Even with numbers and punctuation, a hacking algorithm would solve that pretty much instantaneously.

Notice what has just happened here. You are testing your assumption that the initial acceptance criteria represent all of the constraints that make a secure password. At each step, you used a different example to verify your understanding of the various rules. Now you seem to have found another requirement that you need to represent: dictionary words should be avoided. You decide to push this further:

You: How about "SeagullHedgehog"?

Raj: That would be better.

You: But there are no numbers or punctuation marks in it.

Raj: Sure, that would make it better. But it's still a random sequence of words, which would be pretty hard to crack.

You: How about "SeagullHedgehogCatapult"?

Raj: Pretty much uncrackable.

To keep track of these cases, Susan the tester has been noting a simple table of examples to use for her tests. Here's what she has so far:

Password	Secure
secret	No
password	No
password1	No
SeagullHedgehog	Yes
SeagullHedgehogCatapult	Yes